

2026 STM32

1. 中断过程

中断触发，NVIC中断优先级判断→内核自动保存上下文→执行中断服务函数→恢复上下文并返回主程序

1.中断触发，NVIC中断优先级判断

检查中断使能，进行优先级比较，向内核发出中断响应信号

2.内核自动保存上下文

(1) 暂停当前正在执行的主程序，将核心寄存器（R0-R3、R12、LR、PC、xPSR）依次压入**当前栈（MSP/PSP）**；

(2) 从**中断向量表**中查找对应中断的服务函数入口地址

(3) 内核将**PC（程序计数器）**跳转到该入口地址，开始执行中断服务函数

3.执行中断服务函数

清除中断挂起位，执行中断处理逻辑

4.恢复上下文并返回主程序

(1) 执行 **BX LR** 指令，触发内核硬件自动恢复上下文；

(2) 从栈中依次弹出之前保存的寄存器（R0-R3、R12、LR、PC、xPSR）；

(3) PC 跳回主程序中中断前的执行位置，继续运行主程序；

(4) 若有挂起的低优先级中断，此时 NVIC 会重新判断，触发下一个中断响应

2. CPU寄存器

1.通用寄存器R0-R15

R12（IP）：临时寄存器

R13（SP）：堆栈指针寄存器，MSP主堆栈指针寄存器，PSP进程堆栈指针寄存器

R14（LR）：链接寄存器，保存函数、异常返回地址

函数跳转时硬件自动保存PC的下一条指令地址到LR

R15（PC）：程序计数器，指向下一条要执行的指令地址

2.状态与控制寄存器

xPSR程序状态寄存器

APSR，应用状态寄存器，保存运算结果标志位（N、Z、C、V、Q）

IPSR，中断程序状态寄存器，保存当前正在执行的异常号（中断号）

EPSR，扩展程序状态寄存器，包含T为（是否Thumb状态）等

CONTROL控制寄存器：控制当前使用堆栈（MSP/PSP）和特权级（特权/非特权）

PRIMASK主中断屏蔽寄存器：置1屏蔽所有可屏蔽中断（仅保留NMI和HardFault）

FAULTMASK故障中断屏蔽控制器：置1屏蔽除NMI外所有中断

BASEPRI基优先级寄存器：设定低于该值的中断不会响应

3.系统控制与状态寄存器

4.NVIC寄存器

5.调试与系统定时寄存器 SysTick

3. 操作模式和特权级别

1.线程模式和handler模式：区别执行的普通应用程序的代码和异常服务例程的代码

2.特权级和用户级：提供一种存储器访问的保护机制

4. IIC

1.通信协议

（1）物理层：两根线（**时钟线SCL和数据线SDA**，半双工双向传输，空闲时SCL和SDA都为高电平），支持多主多从（总线上可以挂载多个从设备，**每个从设备有唯一的7/10位地址，主设备通过地址选择通信对象**）

（2）协议层：时序为起始信号→地址帧→数据帧→应答→停止信号

① **起始信号（Start）**：主设备拉低 SDA（SCL 保持高电平），表示通信开始

② **地址帧**：主设备发送 7 位从设备地址 + 1 位读写位（0 写 / 1 读），从设备匹配地址后拉低 SDA 产生**应答（ACK）**

③ **数据帧**：每次传输 8 位数据，发送方发完 1 字节后，接收方必须回 ACK（表示收到）

④ **停止信号（Stop）**：主设备在 SCL 高电平时拉高 SDA，表示通信结束

2.外设驱动

（1）硬件I2C外设：不需要CPU干预，时序固定

（2）软件模拟I2C：时序可控，兼容性好，但是占用CPU资源

（3）关键注意点

① GPIO 模式必须配置为**复用开漏输出（AF_OD）**：I2C 总线是双向的，开漏输出才能实现 SDA 的双向传输

SCL和SDA都需要接上拉电阻

线与功能：只要有一个设备发送低电平时，总线上就表现为低电平

② 时钟速率：**标准模式 100KHz，快速模式 400KHz**，高速模式下可待3.4Mbit/s，需和从设备匹配；

③ 超时处理：实际开发中需加超时判断（如 `while(xxx && timeout--)`），避免程序卡死

5. UART

1.通信协议

（1）物理层：**UART需要三根线（TX，RX，GND共地）**，如果流控需要加RTS/CTS

电平标准TTL，高电平 3.3V/5V（逻辑 1），低电平 0V（逻辑 0）

全双工，但是异步进行

（2）协议层：以帧为单位

空闲位（高电平）

起始位（1位，低电平）→ 数据位（5~8位，常用8位）→ 校验位（0/1位，奇/偶/无）→ 停止位（1/1.5/2位，常用1位）

核心参数（必须双方一致，否则乱码）：

① 波特率：每秒传输的比特数（常用 9600、115200、460800）

② 数据位：有效数据长度，8 位是嵌入式标配（刚好 1 字节，适配 uint8_t）；

③ 校验位：用于检错（嵌入式低功耗 / 简单场景常关闭，靠上层协议检错）；

④ 停止位：帧结束标志，1 位足够，2 位用于低速 / 高干扰场景；

2.外设驱动

（1）STM32 的 USART 外设（如 USART1/2/3）支持**查询、中断、DMA**三种收发方式

（2）外设配置/初始化

配置时钟->GPIO电气属性、模式配置->串口配置

波特率设置，数据格式（字长、停止位、奇偶校验位），硬件数据流控制，收发模式，使能串口/串口接收中断

（3）中断接收

串口接收中断：识别起始位->定时采样8次（波特率相等）->检查停止位->成功后把字节写入接收寄存器->触发RXNE接收完成中断

串口发送：TXE发送寄存器为空->写入到发送寄存器->硬件开始逐位发送（阻塞式）->等待发送寄存器为空

（4）硬件流控制

解决设备发送给数据过快，接收设备出现丢弃或覆盖数据的现象

nRTS引脚用于作为接收方时通知发送方是否做好了接收数据的准备，nCTS引脚用于作为发送方时判断接收方是否做好了接收数据的准备，两个引脚均为低电平有效

(5) USART

SCLK是产生同步的时钟信号，它**配合发送移位寄存器输出**（发送寄存器每移位一次，同步时钟电平就跳变一个周期）。同步时钟信号可以**兼容其它通信协议**，也可以帮助接收方接收数据，**提供自适应波特率**（比如接收设备不确定发送设备的波特率，那么它可以对该时钟的周期进行**测量计算出波特率**）

3. 实战问题

(1) 波特率误差问题

STM32 的 USART 波特率由 `USART_BRR` 寄存器决定（ $BRR = PCLK / baudrate$ ），若 **PCLK（APB 时钟）不是波特率的整数倍，会产生误差**

（如 115200 波特率，PCLK=36MHz 时误差 $\approx 0.16\%$ ，可接受；PCLK=8MHz 时误差 $\approx 3.5\%$ ，会乱码）

解决：调整 MCU 时钟（如用外部晶振），或选择误差小的波特率（如 9600 误差 $< 1\%$ ）

(2) 接收缓冲区溢出

中断接收时，若数据到来速度快于处理速度，缓冲区会溢出，导致数据丢失；

解决：① 增大缓冲区；② **用环形缓冲区（FIFO）**；③ 结合 FreeRTOS 队列（中断中把数据写入队列，任务中解析）

6. SPI

1. 通信协议

(1) 物理层：**有四根线（SCK时钟线，MOSI，MISO，CS片选线，主设备拉低选中某从设备），一主多从，通过 NSS 引脚区分，全双工，收发同时进行**

TTL 电平（3.3V/5V），无需上拉电阻（区别于 I2C）

(2) 协议层

(1) 四种工作方式：配置**输出串行同步时钟极性和相位**

如果CPOL=0，串行同步时钟的空闲状态为低电平；如果CPOL=1，串行同步时钟的空闲状态为高电平

如果CPHA=0，在串行同步时钟的第一个跳变沿（上升或下降）数据被采样；如果CPHA=1，在串行同步时钟的第二个跳变沿被采样

(2) 传输时序

主设备拉低 NSS，选中目标从设备

主机产生时钟信号，将数据一位一位地从MOSI信号线上发送到从机，从机通过MISO将数据发送给从机（全双工）

传输完成后，主设备拉高 NSS，释放从设备

(3) 数据位宽：常用 8 位（1 字节），部分外设支持 16 位（如 ADC、DAC），传输时高位先行 / 低位先行需与从设备匹配

2. 外设驱动

(1) 支持硬件 SPI 外设（如 SPI1/SPI2）和软件模拟 SPI（常用于调试）两种方式

(2) 外设初始化

配置时钟->配置GPIO（上拉输入、复用推挽输出）->配置SPI外设

CS使用软件管理方式，工作模式，数据宽度，SPI工作方式（SPOL和SPHA），时钟频率，数据格式，使能SPI

3. 实战问题

(1) 时钟频率匹配问题

从设备有最大时钟频率限制（如 W25Q64 最大支持 80MHz，OLED 最大支持 10MHz），主设备 SCK 频率不能超过该值；

解决：通过 `SPI_BaudRatePrescaler` 调整分频，如 STM32 PCLK=36MHz 时，分频 2→18MHz，分频 4→9MHz。

(2) 片选信号时序问题

CS 信号必须在 SCK 开始前拉低，SCK 结束后拉高，且拉低 / 拉高后需加短暂延时（>1μs），否则从设备未就绪；

多从设备场景：硬件片选（NSS 引脚）优先级高于软件片选，避免多个从设备同时响应

(3) 数据传输方向问题

SPI 是全双工，若仅读 / 仅写，需发送“空数据”（如 0xFF）来触发时钟，才能接收从设备数据；

例：仅读取 Flash 数据时，需发送 0xFF，同时接收 Flash 返回的字节

7. W25Q64

1. (8MB=64Mbit, 000000H-7FFFFFFH, 24位寻址)

128个块（每个块64KB，地址前两位）->每个块16个扇区（每个扇区4KB，占低四位地址）->每个扇区16页（每页256字节，地址变化为最低一位）->存储器以字节为单位

页地址锁存器，字节地址缓存器，**页缓冲区（RAM，256字节，即一个扇区）**

最小擦除单位为1个扇区（4KB，低四位地址）

连续写入多字节时，最多写入一页的数据（256字节，最低一位）

防止跨页：判断 起始地址+数据长度 是否超出本页，若超出就写在下一页，并更新数据结尾位置

2. 页编程（0x000000~0x7FFFFF）

写入之间需要擦除，写入的地址范围不能跨页，写入操作结束后需要等待忙完信号

写使能->交换发送页编程指令->写地址高8位、中8位、低8位->写入的数据->终止信号->等待忙，更新数据结尾位置

页擦除（4KB）

读数据：不能在忙状态下读数据

8. EXTI

监测指定GPIO口的电平信号，当其指定的GPIO口产生电平变化时，EXTI将立即向NVIC发出中断申请，经过NVIC裁决后即可中断CPU主程序，使CPU执行EXTI对应的中断程序

支持的触发方式：上升沿/下降沿/双边沿/软件触发

支持所有GPIO口触发，但相同的Pin不能同时触发中断

通道数：16Pin、PVD输出、RTC闹钟、USB唤醒、以太网唤醒

触发响应方式：中断响应/事件响应

AFIO完成复用引脚重映射和中断引脚选择

9. GPIO

通用输入输出口

配置GPIO的端口配置寄存器配置8种输入输出模式

输入：**浮空**（引脚悬空）、**上拉**（内部连接上拉电阻）、**下拉**、**模拟**（GPIO无效，引脚直接接入内部ADC）

输出：**开漏输出**（高电平为高阻态），**推挽输出**（高电平接VDD），**复用开漏**（由片上外设控制），**复用推挽**

10. MPU 内存保护单元

给没有操作系统、或跑 RTOS 的 MCU 做内存保护，不让程序乱踩内存、非法访问、栈溢出、外设越权写，用于 Cortex-M 系列

1.只做保护，不做地址转换

2.把内存分成多个**区域（region）**，每个区域设置：

- 可读 / 可写 / 可执行
- 特权级 / 用户级权限
- 缓存策略

越权访问直接触发**MemManage Fault / HardFault**